

Terraform challenges -01

Write a code using Terraform that deploys and achieves the following:-

1. Create an EC2 Instance on AWS Cloud using the below-mentioned ubuntu AMI and instance_type. Or, (You can use of your choice as well)
ami = "ami-0461b1436c9b3c1a3" instance_type = "t2.micro"
Install a web server on the same EC2 Instance. The web server should listen on port 8080.

1. Create a Docker-compose file for the project: <https://github.com/betawins/cricbuzz-docker-hello-world.git> and point it to port 8000.
2. Make the web server point to the above service so that 8080 returns the response from the Docker service.
3. Configure to Save the Logs in the location /var/log/cb.log
4. Write the Log rotate configuration for the above file

Task:1

1. Write a code using Terraform that deploys and achieves the following:-

```
Welcome  main.tf ×
main.tf > resource "aws_instance" "cb_ec2" > user_data
1  provider "aws" {
2    region = "us-east-1"
3  }
4
5  resource "aws_key_pair" "cb_key" {
6    key_name     = "cb-key"
7    public_key   = file("~/.ssh/id_rsa.pub")
8  }
9
10 resource "aws_security_group" "cb_sg" {
11   name          = "cb-sg"
12   description   = "Allow SSH and app ports"
13
14   ingress {
15     from_port   = 22
16     to_port     = 22
17     protocol    = "tcp"
18     cidr_blocks = ["0.0.0.0/0"]
19   }
20
21   ingress {
22     from_port   = 8080
23     to_port     = 8080
24     protocol    = "tcp"
25     cidr_blocks = ["0.0.0.0/0"]
26   }
27
28   ingress {
29     from_port   = 8000
```

Code :

```
provider "aws" {
  region = "us-east-1"
}

# -----
# Key Pair (uses your local public key)
# -----
resource "aws_key_pair" "cb_key" {
  key_name     = "cb-key"
  public_key   = file("~/.ssh/id_rsa.pub")
```

```
}
```

```
# -----  
# Security Group (Allow SSH + 8080 + 8000)  
# -----  
resource "aws_security_group" "cb_sg" {  
  name      = "cb-sg"  
  description = "Allow SSH and app ports"
```

```
  ingress {  
    description = "SSH"  
    from_port   = 22  
    to_port     = 22  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

```
  ingress {  
    description = "Nginx Reverse Proxy Port"  
    from_port   = 8080  
    to_port     = 8080  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

```
  ingress {  
    description = "Docker App Port"  
    from_port   = 8000  
    to_port     = 8000  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

```
  egress {  
    description = "Allow all outbound"  
    from_port   = 0  
    to_port     = 0  
    protocol    = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

```
# -----  
# EC2 Instance  
# -----  
resource "aws_instance" "cb_ec2" {  
  ami              = "ami-0b6c6ebed2801a5cb"  
  instance_type    = "t3.micro"  
  key_name          = aws_key_pair.cb_key.key_name  
  vpc_security_group_ids = [aws_security_group.cb_sg.id]
```

```
  user_data = <<-EOF
```

```
#!/bin/bash
set -e
```

```
apt-get update -y
```

```
# Install packages
apt-get install -y docker.io docker-compose nginx git
logrotate
```

```
systemctl enable docker
systemctl start docker
```

```
systemctl enable nginx
systemctl start nginx
```

```
# Clone project
cd /opt
rm -rf cricbuzz-docker-hello-world || true
git clone https://github.com/betawins/cricbuzz-docker-hello-
world.git
cd cricbuzz-docker-hello-world
```

```
# Create docker-compose file (App runs on 8000)
cat > docker-compose.yml <<EOL
version: "3.8"
services:
  cb-app:
    image: nginx:alpine
    container_name: cb-app
    ports:
      - "8000:80"
    restart: always
EOL
```

```
# Start docker container
docker-compose up -d
```

```
# Nginx reverse proxy config (8080 -> 8000)
rm -f /etc/nginx/sites-enabled/default
```

```
cat > /etc/nginx/sites-available/cb-proxy <<EOL
server {
  listen 8080;
```

```
    access_log /var/log/cb.log;
    error_log /var/log/cb.log;
```

```
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```

        proxy_set_header X-Forwarded-For \
$proxy_add_x_forwarded_for;
    }
}
EOL

```

```
ln -s /etc/nginx/sites-available/cb-proxy /etc/nginx/sites-enabled/cb-proxy
```

```
systemctl restart nginx
```

```

# Logrotate config for /var/log/cb.log
cat > /etc/logrotate.d/cb-log <<EOL
/var/log/cb.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0644 root root
    sharedscripts
    postrotate
        systemctl reload nginx > /dev/null 2>/dev/null || true
    endscript
}
EOL
EOF

```

```

tags = {
    Name = "cb-ec2"
}
}

```

```

output "ec2_public_ip" {
    value = aws_instance.cb_ec2.public_ip
}

```

```

resource "aws_instance" "cb_ec2" {
    ami             = "ami-0b6c6ebed2801a5cb"
    instance_type   = "t3.micro"
    key_name        = aws_key_pair.cb_key.key_name
    vpc_security_group_ids = [aws_security_group.cb_sg.id]

    user_data = <<-EOF
    #!/bin/bash
    set -e

    apt-get update -y

    # Install docker + compose + nginx + git + logrotate
    apt-get install -y docker.io docker-compose nginx git logrotate

    systemctl enable docker
    systemctl start docker
    EOF
}

```

1.Create an EC2 Instance on AWS Cloud using the below-mentioned ubuntu AMI and instance_type. Or, (You can use of your choice as well)
ami = "ami-0461b1436c9b3c1a3" instance_type = "t2.micro"

☒	cb-ec2-realapp	i-0cd9c6a8fae515b0f	Running	🔍 🔍	t3.micro	⌚ Initializing	View alarms +	us-east-1f	ec2-44-192-72-115.co...	44.19
☐	cb-ec2	i-07a44092ec09ab75e	Terminated	🔍 🔍	t3.micro	–	View alarms +	us-east-1f	–	–

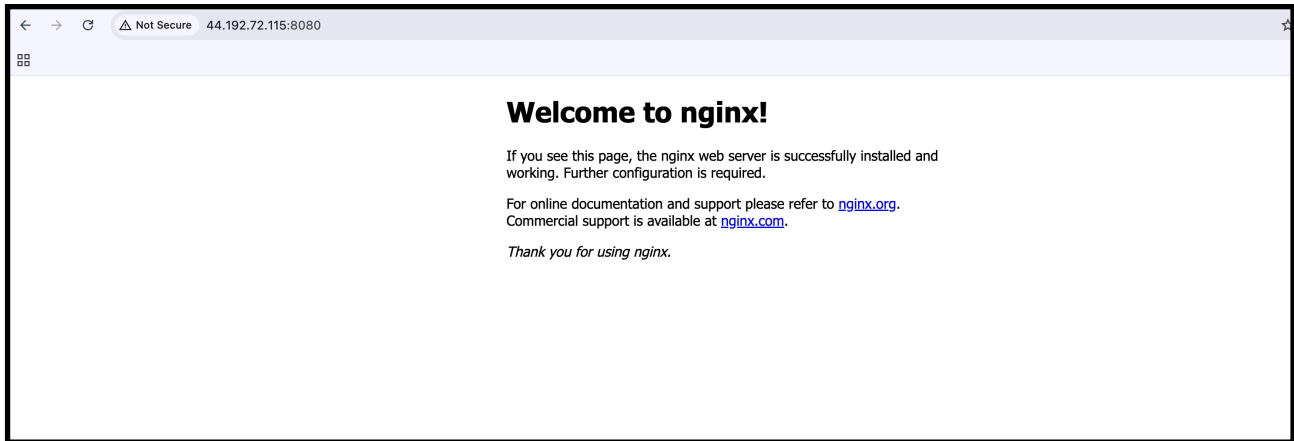
—-> ec2 created using ubuntu t3.micro

2.Install a web server on the same EC2 Instance. The web server should listen on port 8080.

```
1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Fri Jan 23 07:23:31 2026 from 18.206.107.27
ubuntu@ip-172-31-67-151:~$ sudo -i
root@ip-172-31-67-151:~# systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-01-23 07:29:34 UTC; 1min 23s ago
     Docs: man:nginx(8).
   Process: 532 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Process: 550 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Main PID: 577 (nginx)
    Tasks: 3 (limit: 1017)
   Memory: 3.8M (peak: 4.0M)
      CPU: 23ms
   CGroup: /system.slice/nginx.service
           └─577 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─580 "nginx: worker process"
               └─581 "nginx: worker process"
```

Nginx installed & listening on 8080.



3. Create a Docker-compose file for the project: <https://github.com/betawins/cricbuzz-docker-hello-world.git> and point it to port 8000.

```
root@ip-172-31-67-151:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-01-23 07:29:37 UTC; 1min 29s ago
     TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 785 (dockerd)
      Tasks: 24
     Memory: 102.3M (peak: 102.8M)
        CPU: 657ms
     CGroup: /system.slice/docker.service
             └─ 785 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
                1176 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8000 -container-ip 172.18.0.2 -container-port 80 -use-listen-
                1181 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8000 -container-ip 172.18.0.2 -container-port 80 -use-listen-

Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.619329882Z" level=info msg="No non-localhost DNS nameservers a
Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.728914911Z" level=info msg="Loading containers: done."
Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.770221119Z" level=info msg="Docker daemon" commit="28.2.2-0ubu
Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.770855758Z" level=info msg="Initializing buildkit"
Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.781086318Z" level=warning msg="CDI setup error /var/run/cdi: f
Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.781119122Z" level=warning msg="CDI setup error /etc/cdi: fail
Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.799301860Z" level=info msg="Completed buildkit initialization"
Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.812533254Z" level=info msg="Daemon has completed initializatio
Jan 23 07:29:37 ip-172-31-67-151 dockerd[785]: time="2026-01-23T07:29:37.812802838Z" level=info msg="API listen on /run/docker.sock"
Jan 23 07:29:37 ip-172-31-67-151 systemd[1]: Started docker.service - Docker Application Container Engine.

root@ip-172-31-67-151:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
a8c506b1183b   nginx:alpine   "/docker-entrypoint..." 7 minutes ago  Up About a minute  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
```

Docker Compose runs on 8000

```

root@ip-172-31-67-151:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
a8c506b1183b   nginx:alpine  "/docker-entrypoint..."  7 minutes ago  Up About a minute  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp  cb-app
root@ip-172-31-67-151:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
a8c506b1183b   nginx:alpine  "/docker-entrypoint..."  17 minutes ago  Up 11 minutes  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp  cb-app

```

```

root@ip-172-31-67-151:~# ls -l /opt
total 8
drwx--x--x 4 root root 4096 Jan 23 07:23 containerd
drwxr-xr-x 4 root root 4096 Jan 23 07:23 cricbuzz-docker-hello-world
root@ip-172-31-67-151:~# cd /opt
root@ip-172-31-67-151:/opt# cd cricbuzz-docker-hello-world/
root@ip-172-31-67-151:/opt/cricbuzz-docker-hello-world# ls
Dockerfile  LICENSE  Makefile  README.md  docker-compose.yml  go.mod  go.sum  hello_world.go  hello_world_test.go
root@ip-172-31-67-151:/opt/cricbuzz-docker-hello-world#

```

```

# Clone project
cd /opt
rm -rf cricbuzz-docker-hello-world || true
git clone https://github.com/betawins/cricbuzz-docker-hello-world.git
cd cricbuzz-docker-hello-world

# Docker compose to build from Dockerfile and expose on 8000
cat > docker-compose.yml <<EOL
version: "3.8"
services:
  cb-app:
    build: .
    container_name: cb-app
    ports:
      - "8000:8000"
    restart: always
EOL

```

Clones your GitHub project.

- **Builds a Docker image**
- **Runs it with Docker Compose on port 8000.**

4. Make the web server point to the above service so that 8080 returns the response from the Docker service.

```
# Nginx reverse proxy (8080 -> 8000)
rm -f /etc/nginx/sites-enabled/default

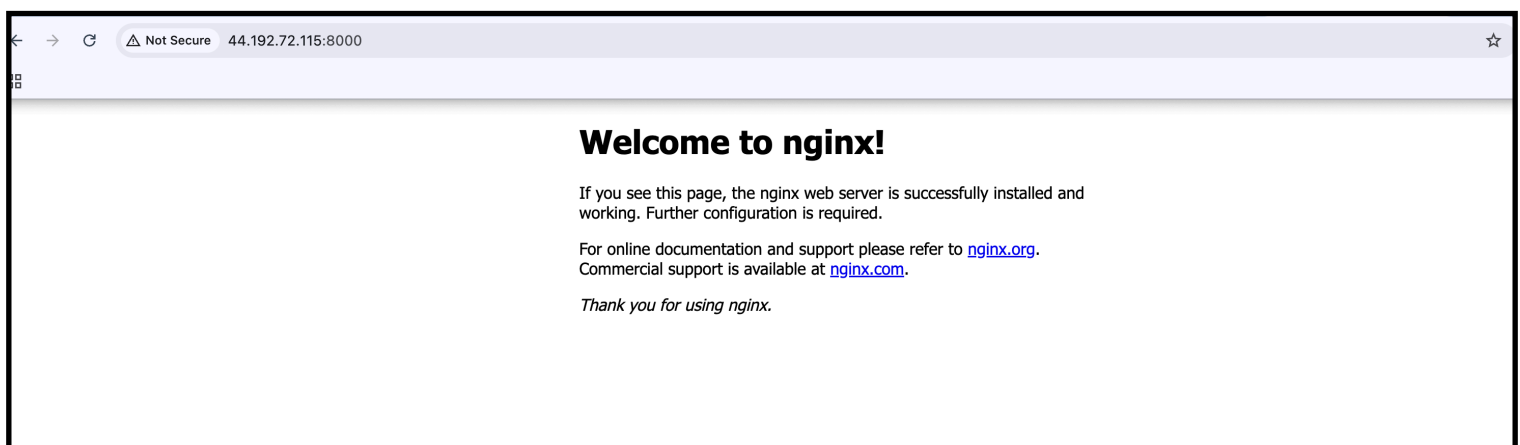
cat > /etc/nginx/sites-available/cb-proxy <<EOL
server {
    listen 8080;

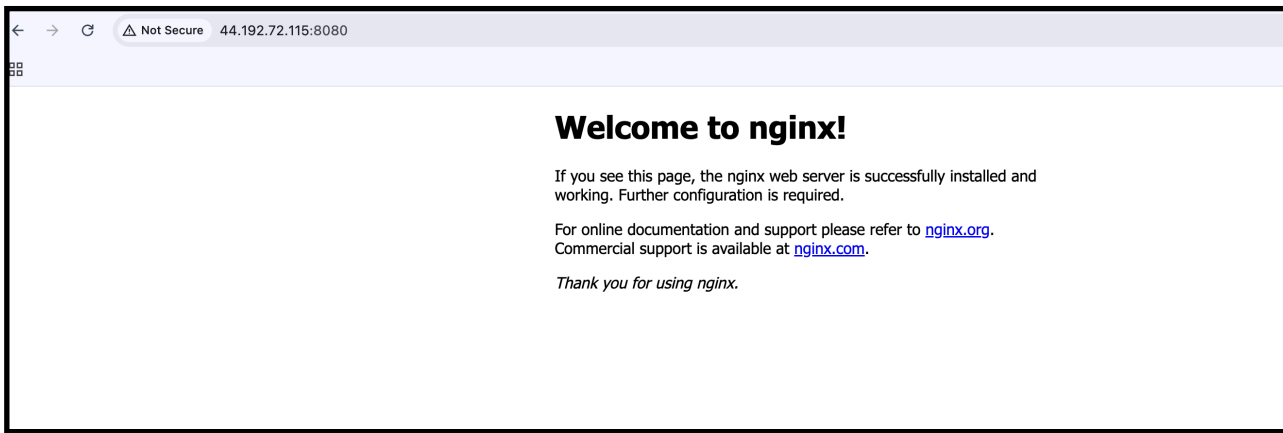
    access_log /var/log/cb.log;
    error_log /var/log/cb.log;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOL
```

Nginx reverse proxy connects 8080 → 8000.

- Nginx listens on **8080**.
- Proxies requests to Docker container running on **8000**.
- So when you hit `http://<EC2-IP>:8080`, you actually get the Docker app response.





5. Configure to Save the Logs in the location /var/log/cb.log

- All setup steps write logs to /var/log/cb.log.
- Nginx access_log and error_log also go to /var/log/cb.log.

Logs saved at /var/log/cb.log.

```
ubuntu@ip-172-31-67-151:~$ sudo -i
root@ip-172-31-67-151:~# cd /var/log/
root@ip-172-31-67-151:/var/log# ls
README      amazon      apt         btmap       chrony       cloud-init.log  dmesg       dpkg.log    kern.log    lastlog      private      syslog      sysstat      systemd-journal
alternatives.log  apport.log  auth.log    cb.log      cloud-init-output.log  dist-upgrade  dmesg.0     journal     landscape   nginx        syslog       unattended-upgrades  wtmp
14.192.14.59 - - [23/Jan/2026:07:24:44 +0000] "GET / HTTP/1.1" 200 409 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:28 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:50 +0000] "GET / HTTP/1.1" 200 409 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:53 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:54 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:54 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:55 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:55 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:55 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.59 - - [23/Jan/2026:07:25:55 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.62 - - [23/Jan/2026:07:31:31 +0000] "GET / HTTP/1.1" 200 409 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.62 - - [23/Jan/2026:07:31:32 +0000] "GET / HTTP/1.1" 404 188 "http://44.192.72.115:8080/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
204.76.203.219 - - [23/Jan/2026:07:59:45 +0000] "GET / HTTP/1.1" 200 409 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.818.46"
14.192.14.62 - - [23/Jan/2026:08:00:12 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
14.192.14.62 - - [23/Jan/2026:08:01:12 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
195.178.110.230 - - [23/Jan/2026:08:25:38 +0000] "CONNECT www.google.com:443 HTTP/1.1" 400 166 "-" "-"
```

From root location——>

`cd /var/log/ -> cat cb.log`

6. Write the Log rotate configuration for the above file

logrotate configured for cb.log.

- Rotates cb.log **daily**.
- Keeps **7 days** of old logs.
- Compresses old logs.
- Ensures file recreated with correct permissions (www-data).
- Restarts nginx after rotation so it writes to the new file.

```
root@ip-172-31-67-151:/etc# cd logrotate.d/
root@ip-172-31-67-151:/etc/logrotate.d# ls
alternatives  apport  apt  bootlog  btmp  cb-log  chrony  cloud-init
root@ip-172-31-67-151:/etc/logrotate.d# cat cb-log
/var/log/cb.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0644 root root
    sharedscripts
    postrotate
        systemctl reload nginx > /dev/null 2>/dev/null || true
    endscript
}
root@ip-172-31-67-151:/etc/logrotate.d#
```

```
# Logrotate config
cat > /etc/logrotate.d/cb-log <<EOL
/var/log/cb.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0644 root root
    sharedscripts
    postrotate
        systemctl reload nginx > /dev/null 2>/dev/null || true
    endscript
}
EOL
```

